

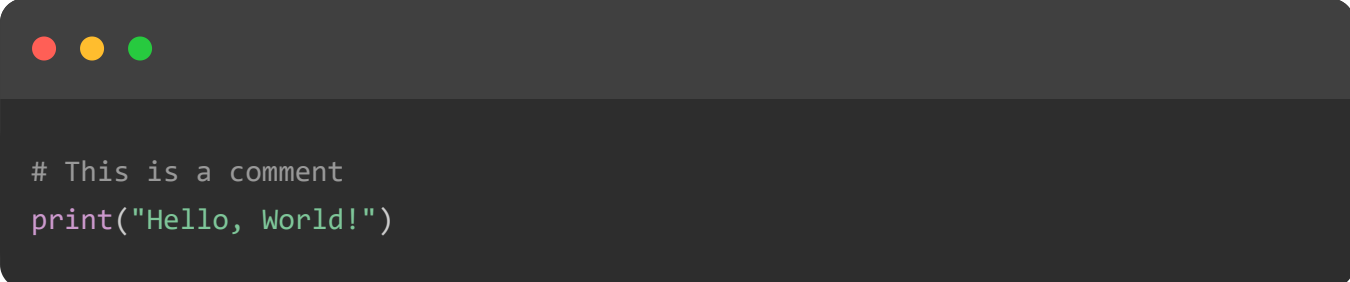
Python - Comments

Python Comments

Python comments are programmer-readable explanation or annotations in the Python source code. They are added with the purpose of making the source code easier for humans to understand, and are ignored by Python interpreter. Comments enhance the readability of the code and help the programmers to understand the code very carefully.

Example

If we execute the code given below, the output produced will simply print "Hello, World!" to the console, as comments are ignored by the Python interpreter and do not affect the execution of the program –



```
# This is a comment  
print("Hello, World!")
```

Python supports three types of comments as shown below –

- Single-line comments
- Multi-line comments
- Docstring Comments

Single Line Comments in Python

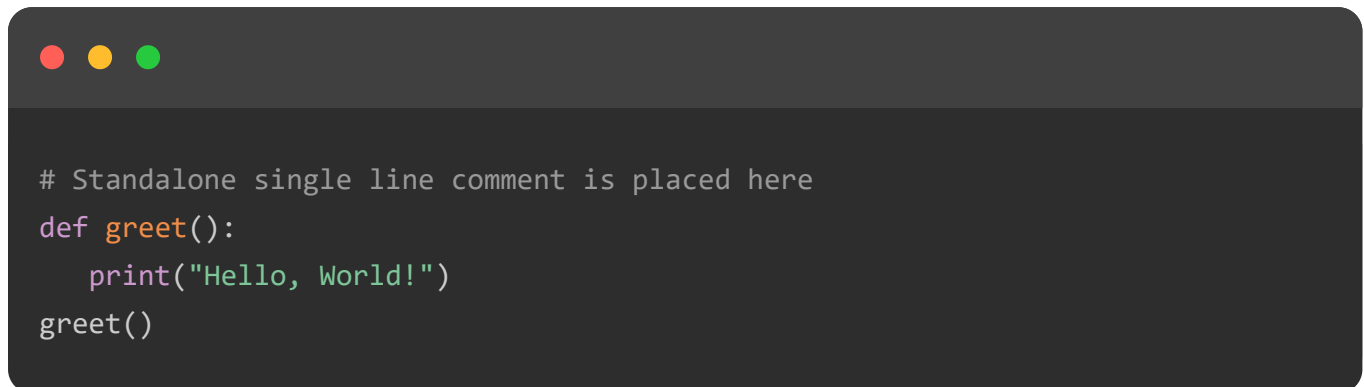
Single-line comments in Python start with a hash symbol (#) and extend to the end of the line. They are used to provide short explanations or notes about the code. They can

be placed on their own line above the code they describe, or at the end of a line of code (known as an inline comment) to provide context or clarification about that specific line.

Example: Standalone Single-Line Comment

A standalone single-line comment is a comment that occupies an entire line by itself, starting with a hash symbol (#). It is placed above the code it describes or annotates.

In this example, the standalone single-line comment is placed above the "greet" function "_

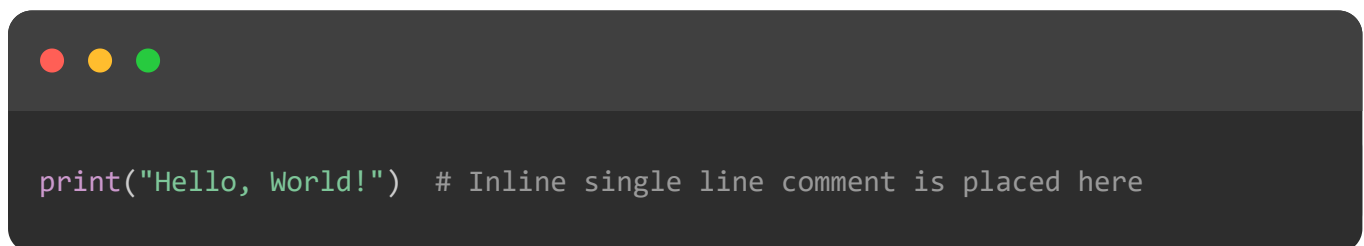


```
# Standalone single line comment is placed here
def greet():
    print("Hello, World!")
greet()
```

Example: Inline Single-Line Comment

An inline single-line comment is a comment that appears on the same line as a piece of code, following the code and preceded by a hash symbol (#).

In here, the inline single-line comment follows the **print("Hello, World!")** statement –



```
print("Hello, World!") # Inline single line comment is placed here
```

Multi Line Comments in Python

In Python, multi-line comments are used to provide longer explanations or notes that span multiple lines. While Python does not have a specific syntax for multi-line comments, there are two common ways to achieve this: consecutive single-line comments and triple-quoted strings –

Consecutive Single-Line Comments

Consecutive single-line comments refers to using the hash symbol (#) at the beginning of each line. This method is often used for longer explanations or to section off parts of the code.

Example

In this example, multiple lines of comments are used to explain the purpose and logic of the factorial function –

```
# This function calculates the factorial of a number
# using an iterative approach. The factorial of a number
# n is the product of all positive integers less than or
# equal to n. For example, factorial(5) is 5*4*3*2*1 = 120.
def factorial(n):
    if n < 0:
        return "Factorial is not defined for negative numbers"
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

number = 5
print(f"The factorial of {number} is {factorial(number)}")
```

Multi Line Comment Using Triple Quoted Strings

We can use triple-quoted strings (""" or """) to create multi-line comments. These strings are technically string literals but can be used as comments if they are not assigned to any variable or used in expressions.

This pattern is often used for block comments or when documenting sections of code that require detailed explanations.

Example

Here, the triple-quoted string provides a detailed explanation of the "gcd" function, describing its purpose and the algorithm used –

```
"""
This function calculates the greatest common divisor (GCD)
of two numbers using the Euclidean algorithm. The GCD of
two numbers is the largest number that divides both of them
"""
```

```
without leaving a remainder.
"""
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

result = gcd(48, 18)
print("The GCD of 48 and 18 is:", result)
```

Using Comments for Documentation

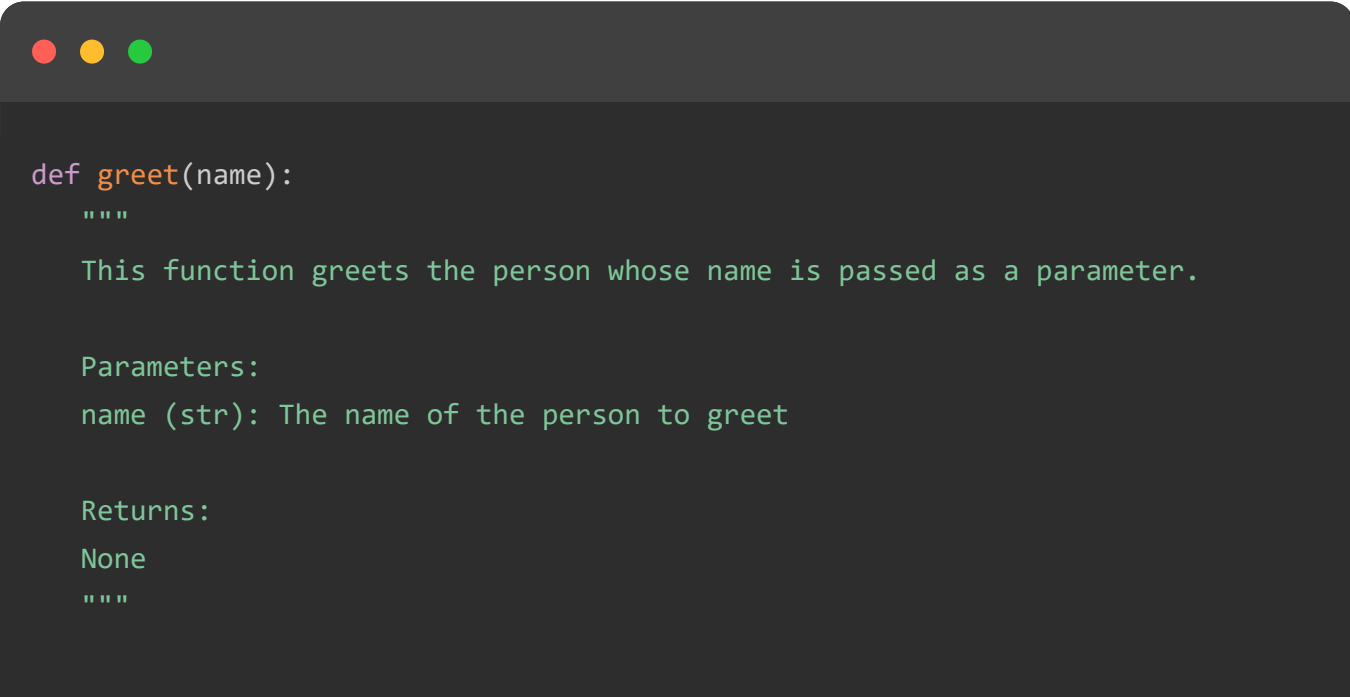
In Python, documentation comments, also known as docstrings, provide a way to incorporate documentation within your code. This can be useful for explaining the purpose and usage of modules, classes, functions, and methods. Effective use of documentation comments helps other developers understand your code and its purpose without needing to read through all the details of the implementation.

Python Docstrings

In Python, docstrings are a special type of comment that is used to document modules, classes, functions, and methods. They are written using triple quotes (""" or ''') and are placed immediately after the definition of the entity they document.

Docstrings can be accessed programmatically, making them an integral part of Python's built-in documentation tools.

Example of a Function Docstring



```
def greet(name):
    """
    This function greets the person whose name is passed as a parameter.

    Parameters:
    name (str): The name of the person to greet

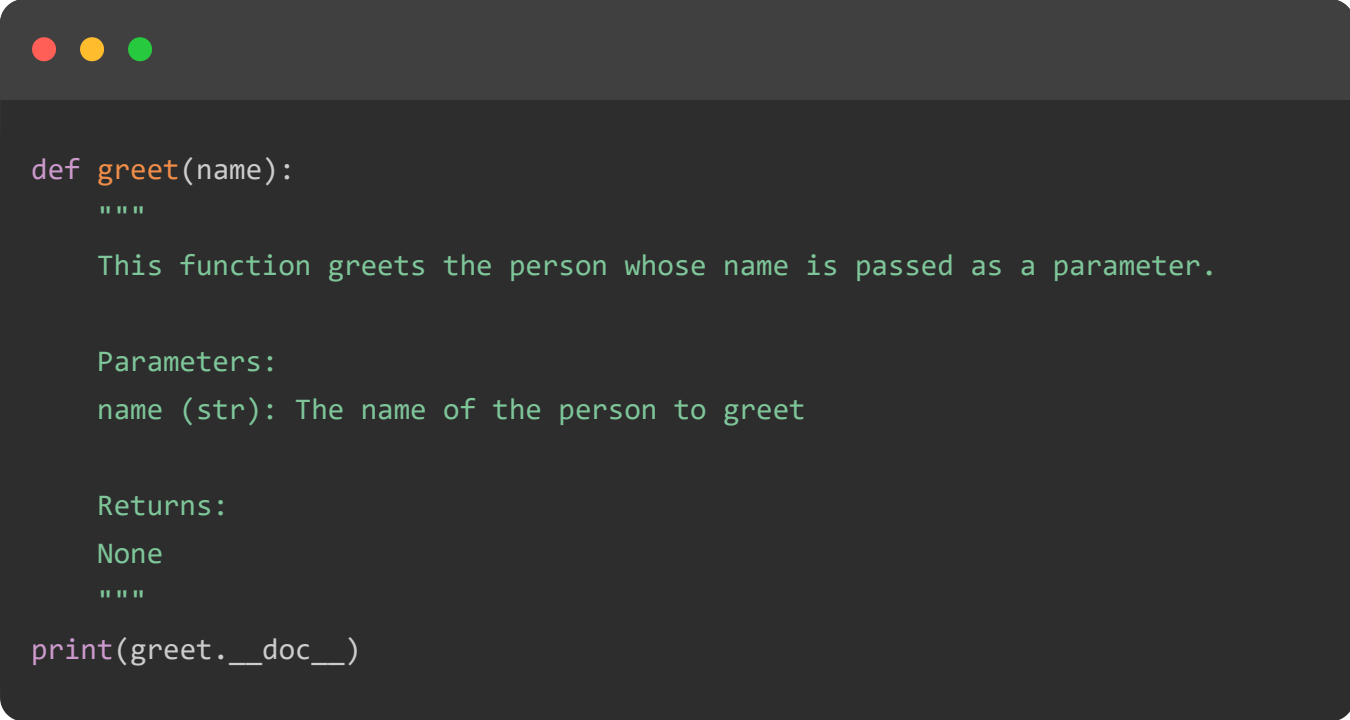
    Returns:
    None
    """
```

```
print(f"Hello, {name}!")  
greet("Alice")
```

Accessing Docstrings

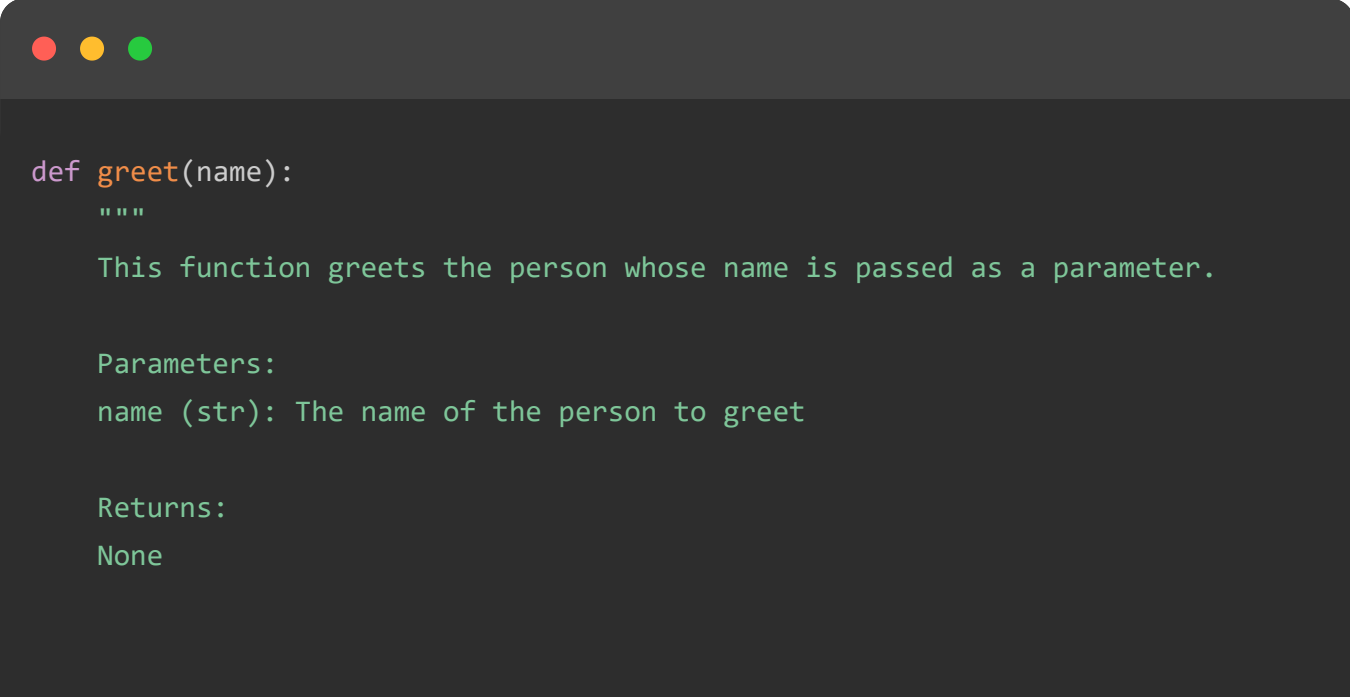
Docstrings can be accessed using the `.__doc__` attribute or the `help()` function. This makes it easy to view the documentation for any module, class, function, or method directly from the interactive Python shell or within the code.

Example: Using the `.__doc__` attribute



```
def greet(name):  
    """  
    This function greets the person whose name is passed as a parameter.  
  
    Parameters:  
    name (str): The name of the person to greet  
  
    Returns:  
    None  
    """  
  
print(greet.__doc__)
```

Example: Using the `help()` Function



```
def greet(name):  
    """  
    This function greets the person whose name is passed as a parameter.  
  
    Parameters:  
    name (str): The name of the person to greet  
  
    Returns:  
    None  
    """
```

```
help(greet)
```

TOP TUTORIALS

Python Tutorial
Java Tutorial
C++ Tutorial
C Programming Tutorial
C# Tutorial
PHP Tutorial
R Tutorial
HTML Tutorial
CSS Tutorial
JavaScript Tutorial
SQL Tutorial

TRENDING TECHNOLOGIES

Cloud Computing Tutorial
Amazon Web Services Tutorial
Microsoft Azure Tutorial
Git Tutorial
Ethical Hacking Tutorial
Docker Tutorial
Kubernetes Tutorial
DSA Tutorial
Spring Boot Tutorial
SDLC Tutorial
Unix Tutorial

CERTIFICATIONS

Business Analytics Certification
Java & Spring Boot Advanced Certification
Data Science Advanced Certification

Cloud Computing And DevOps
Advanced Certification In Business Analytics
Artificial Intelligence And Machine Learning
DevOps Certification
Game Development Certification
Front-End Developer Certification
AWS Certification Training



Chapters ▾

Categories ≡

COMPIERS & EDITORS

Online Java Compiler
Online Python Compiler
Online Go Compiler
Online C Compiler
Online C++ Compiler
Online C# Compiler
Online PHP Compiler
Online MATLAB Compiler
Online Bash Terminal
Online SQL Compiler
Online Html Editor

[ABOUT US](#) | [OUR TEAM](#) | [CAREERS](#) | [JOBS](#) | [CONTACT US](#) | [TERMS OF USE](#) |
[PRIVACY POLICY](#) | [REFUND POLICY](#) | [COOKIES POLICY](#) | [FAQ'S](#)



Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2026. All Rights Reserved.